



Lecture 3 — Claude Code for Brownfield Development

AI-Assisted Software Development and Intelligent Applications

Today's Agenda

1. What Claude Code loads automatically vs. what you have to reference
2. `.claude/rules/`, Skills, and settings — the current toolkit
3. Driving development in a codebase you didn't write
4. Merge conflicts — why they happen, how to resolve one
5. **Activity** — Tower Defense Lite: document it, then survive a conflict

Pre-Class Checklist — Start the Quiz

Before we start — did you complete the reading?

- **Claude Code Overview** — prompting strategies, navigating unfamiliar code
- **How Claude Remembers Your Project** — CLAUDE.md vs. auto memory, scopes, `.claude/rules/`

Today you'll meet a codebase you've never seen — Tower Defense Lite — and put both readings to work immediately.

Two Memory Systems

Claude Code has **two complementary systems**, both loaded into every session:

	CLAUDE.md	Auto memory
Who writes it	You	Claude
Contains	Instructions, rules	Learnings, patterns
Use for	Conventions, architecture	What Claude would otherwise repeat

Two Memory Systems, Managed One Way

Both CLAUDE.md and auto memory are browsed and edited with `/memory`.

Auto memory means Claude stops making the same mistake twice — without you writing anything down yourself.

CLAUDE.md: Where It Lives

Loaded in this order, broadest to most specific:

Scope	Location	Shared with
Managed policy	IT-deployed	Everyone
User	<code>~/ .claude/CLAUDE.md</code>	You, everywhere
Project	<code>./CLAUDE.md</code>	Team, via git
Local	<code>./CLAUDE.local.md</code>	You, this project

CLAUDE.md: Nested and Local

Nested CLAUDE.md files in subdirectories load **on demand** — only when Claude reads files there, not at launch.

`CLAUDE.local.md` is gitignored — your personal notes for this one project, never shared with teammates.

Auto Memory: Where It Lives

Location	
Auto memory	<code>~/ .claude/projects/<project>/memory/</code>

Per-project — one memory folder per repo, derived from it automatically. Not a fixed path you'd just happen to know, unlike CLAUDE.md's scopes.

What Auto-Loads

Every session, no action needed:

- CLAUDE.md, CLAUDE.local.md
- Auto memory
- `.claude/rules/*.md`
- `settings.json`

What Doesn't — Unless You Ask

These do **nothing on their own**:

- A root-level `RULES.md`
- `SPEC.md`

A freestanding `RULES.md` in your repo isn't a guardrail yet — it's a file Claude has never read, until something references it.

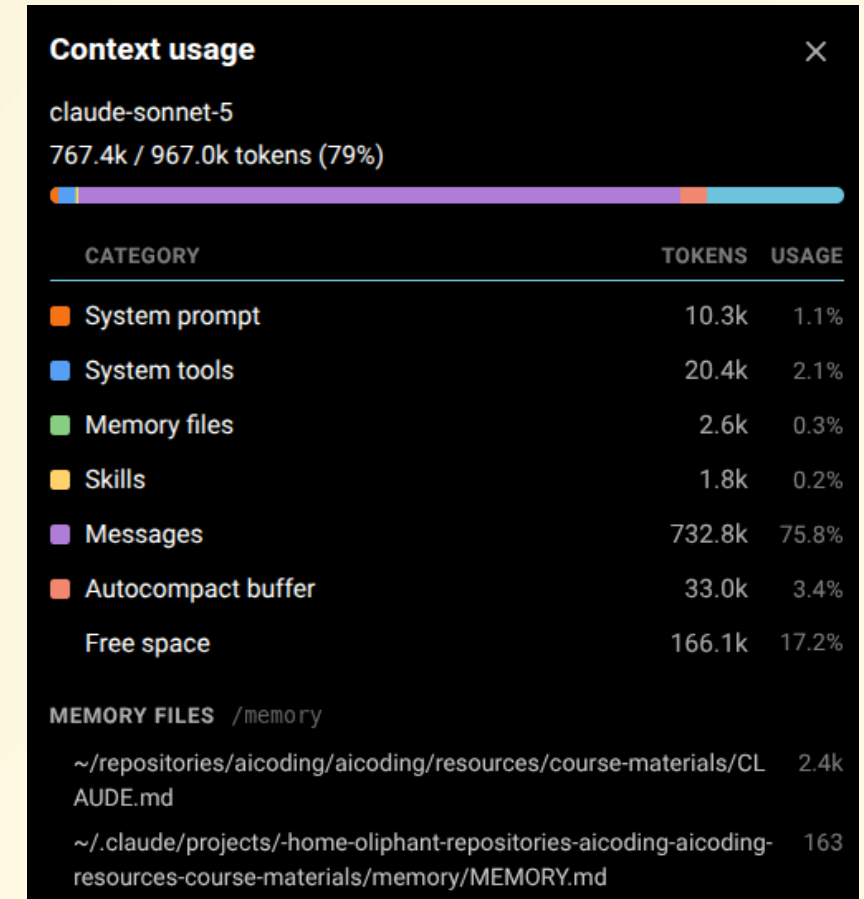
/context — A Real Example

Four small categories are auto-loaded, under 3% combined:

- **System prompt / System tools** — Claude's own instructions and tool definitions
- **Memory files** — CLAUDE.md *and* MEMORY.md, bundled together
- **Skills** — descriptions only; loads fully on invocation

Messages — the conversation itself, by far the largest category, the only one *not* auto-loaded.

MEMORY FILES below names the exact files behind that category.



Making a Reference-Only File Count

`SPEC.md` and root `RULES.md` only take effect if you `@`-import them:

See `@SPEC.md` for the full feature specification.

Without that line in `CLAUDE.md`, `SPEC.md` is just markdown sitting in your repo — Claude never sees it. Today's activity has you prove this to yourself: write `SPEC.md`, then make it actually load.

`.claude/rules/` — Guardrails That Load Themselves

```
---  
paths:  
  - "src/**/*.ts"  
---  
  
# Unit Testing  
Whenever a function is created or modified,  
create accompanying unit tests to verify  
the function operates properly. Follow  
The repos structure for location of the tests.
```

`.claude/rules/` — **Scoped on Purpose**

A rule with no `paths:` loads every session, same priority as CLAUDE.md.

A **scoped** rule (like the unit testing example) only loads when Claude actually touches a matching file — noise stays out of context until it's relevant.

Buzzer Checkpoint

**Launch Question 6 – discuss answers, then
continue**

Skills: Reusable Procedures

Create a `SKILL.md` with instructions, and Claude adds it to its toolkit.

- Claude invokes a skill **automatically when relevant**, or explicitly:
`/skill-name`
- Lives at `.claude/skills/<name>/SKILL.md`
- Good for: a procedure you keep re-explaining, or a checklist that's outgrown CLAUDE.md

Claude Code Workflow Commands

- **`/init`** — generates a starting CLAUDE.md by analyzing your codebase (a first draft, not a finished product)
- **`/memory`** — browse and edit CLAUDE.md/CLAUDE.local.md across scopes, toggle auto memory, open the auto-memory folder
- **`/help`** — This is the source of truth (i.e. maintained compared to outdated slides, websites, etc.)

Settings Scopes (settings.json)

Scope	File	Shared with
User	~/.claude/settings.json	You, all projects
Project	.claude/settings.json	Your team, via git
Local	.claude/settings.local.json	Just you, this project (gitignored)

More specific scope wins. (A fourth, "managed policy" scope exists for org-deployed settings – not something you'll configure yourself.)

Permissions: **allow** / **ask** / **deny**

```
{  
  "permissions": {  
    "allow": ["Bash(npm run test *)"],  
    "ask": ["Bash(git push *)"],  
    "deny": ["Bash(curl *)"]  
  }  
}
```

Three tiers: run silently, prompt first, or refuse outright.

settings.json — Environment Variables

```
{  
  "env": {  
    "NODE_ENV": "development",  
    "API_BASE_URL": "http://localhost:8000"  
  }  
}
```

Every Claude Code session in this project gets these automatically — no `export` in your shell first.

settings.json – Hooks

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          { "type": "command", "command": "npm run format" }
        ]
      }
    ]
  }
}
```

settings.json — Hooks Run Automatically

The example on the last slide runs `npm run format` after every `Edit` or `Write` — no prompting, no opting out.

That's the whole point of a hook, coming up next.

Hooks: Context vs. Enforcement

CLAUDE.md and `.claude/rules/` are context. Claude tries to follow them, but there's no guarantee — vague or conflicting instructions can lose.

Hooks are enforced regardless of what Claude decides. A hook is a script that runs on a tool event (e.g. after every edit) and can actually block an action.

If something *must* happen every time — not just usually — it belongs in a hook, not a CLAUDE.md instruction.

Driving Development in Unfamiliar Code

When opening a codebase you didn't write:

1. **Ask before touching anything** — *"Give me an overview of this structure"*
2. **Trace the behavior you care about** — *"What happens in one frame?"*
3. **Ask about anything that looks strange** first
4. **Then** propose your change and iterate

Why Merge Conflicts Happen

Git can auto-merge two branches **unless they both edited the same lines**. When that happens, it stops and asks you to decide.

```
<<<<<< HEAD
export const ENEMY_SPAWN_INTERVAL_MS = 1200;
=====
export const ENEMY_SPAWN_INTERVAL_MS = 600;
>>>>>> tune/harder-waves
```

Everything above `=====` is your current branch; everything below is the branch you're merging in. Pick one, combine them, or rewrite entirely — then remove the markers so the file is valid again.

Buzzer Checkpoint

**Launch Question 7 – discuss answers, then
continue**

This Is Coming, Starting Project 2

You're practicing solo today by engineering a guaranteed conflict — two branches, same line, different values.

Starting Project 2, this happens **for real**: you and your partner both touch a file, and Git can't guess who's right. The skill is the same either way — read what each side changed, decide (with Claude Code's help if you want), resolve, commit.

Activity — Tower Defense Lite

A TypeScript + p5.js tower defense game — same stack as P1 and Coin Catcher, bigger, and **already complete and working**. Click to place a tower; towers auto-target the enemy that's made the most progress toward your base.

Before touching any code: use Claude Code to explore `project/` — ask it how a Tower picks its target, and why Enemy stores distance traveled instead of x/y and anything else you don't understand.

⚠ Open This Folder Before You Launch Claude Code

This is the lecture where it bites: today's `.claude/rules/` and `.claude/skills/` additions (Before Next Lecture, below) only load if Claude Code's launch folder is this lecture's `project/` — not the repo root, not the lecture folder one level up.

```
code "Lecture 3 - Claude Code for Brownfield Development/project"
```

If your rule or skill silently "does nothing," this is the first thing to check.

Buzzer Checkpoint

**Launch Question 8 — discuss answers, then
continue**

Activity — Parts 1 & 2

Part 1 — Document it: produce a structured **CLAUDE.md** — codebase overview and real conventions, not a restated file tree. Use `/init` as a starting point, then refine it.

Part 2 — Make a reference count: write **SPEC.md** outlining the existing features, then add `See @SPEC.md for the full feature specification.` to **CLAUDE.md** so it's actually pulled into context.

Activity — Part 3: Survive a Merge Conflict

Do these merges locally with `git merge` — not a GitLab merge request.

1. Branch `tune/easier-start`, lengthen `ENEMY_SPAWN_INTERVAL_MS` in `src/Constants.ts`, commit
2. Back on `main`, branch `tune/harder-waves`, shorten the **same line**, commit
3. `git checkout main && git merge tune/easier-start` — clean
4. `git merge tune/harder-waves` — conflicts
5. Resolve it, finish the merge, confirm the game still runs

After each part: `npm run test`, `npm run format`, `npm run lint` — commit — push.

⚠ Don't Erase That Merge Commit Later

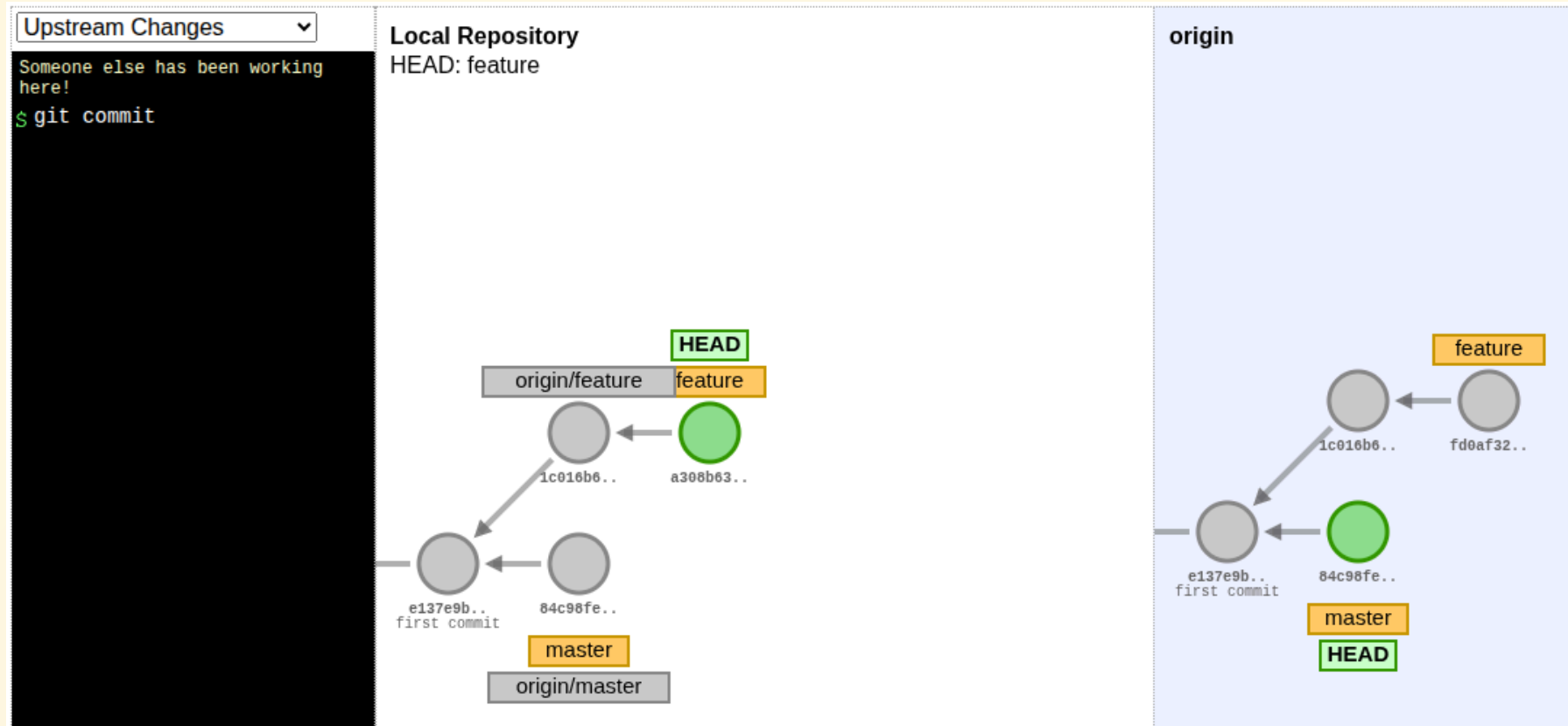
Grading checks this repo for the **merge commit** you just made. **Any time this semester**, don't rebase this repo's history:

```
git rebase ...      # NEVER on this repo - replays commits, deletes merge commits
git pull --rebase   # same thing - never use --rebase here

git merge ...       # safe - this is what keeps your merge commit intact
git pull            # safe - plain pull, no flags, never rewrites history
```

Clicking GitLab's **Update fork** (pulls in new lecture content) already merges safely on its own — nothing to pick, nothing else you need to do. The only way to lose today's merge commit is running `git rebase` (or `git pull --rebase`) yourself, even weeks from now, with no warning.

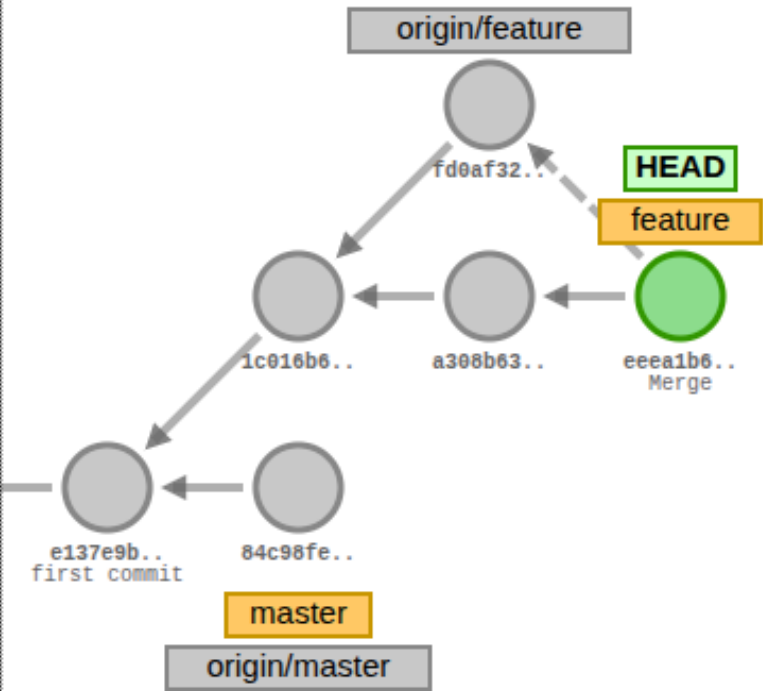
<https://git-school.github.io/visualizing-git/>



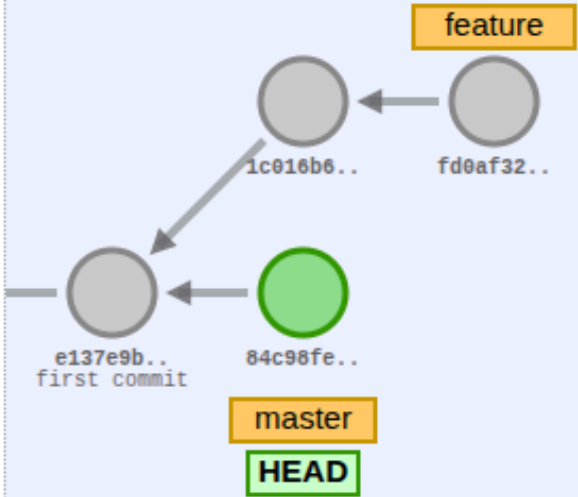
Upstream Changes ▾

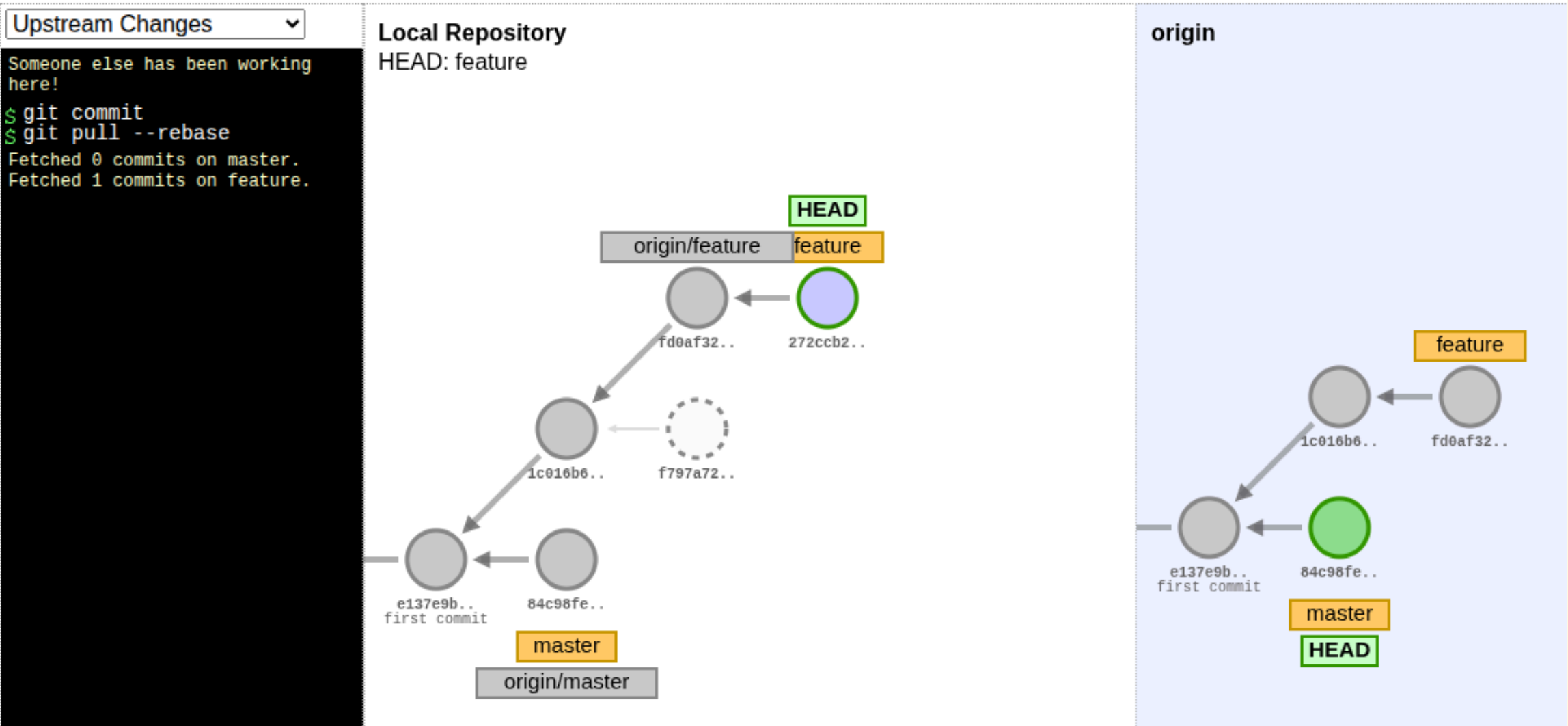
```
Someone else has been working here!  
$ git commit  
$ git pull --merge  
Fetched 0 commits on master.  
Fetched 1 commits on feature.
```

Local Repository
HEAD: feature



origin





Before Next Lecture

Get a head start on **Lecture 4 — The Elephant-Goldfish Model:**

- Read: [*Elephants, Goldfish and the New Golden Age of Software Engineering*](#) (Rensin, 2026) — also posted to Canvas if you hit the Medium paywall
- Using **Lecture 4's** Book Review `project/` — a different codebase from today's — generate a **README hierarchy** with Claude Code: leaves to root, verifying each one before moving up
- Commit it before class — you'll load it straight into Claude Code at the start of Lecture 4



Questions?

Next lecture: The Elephant-Goldfish Model — arrive with the Lecture 4 Book Review README hierarchy already committed (see "Before Next Lecture")